

Research Meta-Policies

Pascal Tohouri

First Posted: 2026-08-12
Current Version: 2026-09-21

1 File Placement Policy

Research is an inherently iterative process. Rarely does one produce a settled draft on the first try. So, given that each file has a set of draft iterations $\{1, 2, \dots, T\}$ and a set of potential locations $\{1, 2, \dots, P\}$, the researcher requires a placement policy

$$\pi : \{1, \dots, T\} \longrightarrow \mathcal{P}(\{1, \dots, P\}),$$

where $\pi(t)$ is the set of locations assigned to iteration t . There are many such policies $\pi \in \Pi$. This note outlines and justifies the author’s preferred policy.

1. If the file contains known (i) spelling/punctuation/grammar/notation errors or (ii) mathematical/logical inconsistencies, and there is no exceptional reason for publication, then retain the file on local devices; otherwise, proceed to (2).

Reason: Known linguistic/logical/mathematical errors risk misleading readers; erroneous work typically does not constitute good science or add value to the literature. Known errors may only pass this filter in special cases, for example, when one wished to crowdsource potential solutions.

2. If the file lacks a potentially novel contribution, then do not place the file in a research repository; consider non-research locations such as Medium, WordPress, or similar platforms. Otherwise, proceed to (3).

Reason: The author reserves research repositories for potentially novel contributions. Replicating existing claims or results in a way that adds no extra information is a wasteful duplication of content that clutters the literature. This idea extends to cases where new information is trivial (e.g., a symmetric restatement of a known result). This filter is weak in the sense that a file containing a high proportion of replicated claims, r , and a low proportion of novel claims, $(1 - r)$, may still proceed to (3). Loosely, for example, one would consider a file with $r < 0.99$ to contain non-replicated or non-trivial claims, although impact, rather than proportion, is often the preferred indicator.

3. If the file contains minor changes relative to the prior “Figshare-1” (F1) iteration, place the file in Github (GH)¹; otherwise, proceed to (4).

Reason: Error-free files containing potentially novel contributions but only marginal changes from prior iterations create high-frequency versioning sequences, in which a single repository may receive, say, 30+ versions of very similar research. This high-frequency sequence is a valuable record of each project’s evolution and thus warrants public storage/time-stamping. Yet, repetitively updating a repository that is widely viewed or linked externally may create version fatigue for readers following the project. My Github is an isolated intermediary appropriate for storing files of this kind.

4. If the file contains major changes relative to the prior “Figshare-1” iteration but contains neither a research paper nor relevant supporting materials, place the file in GH and “Figshare-1”; otherwise, proceed to (5).

¹(Previously Figshare-2.)

Reason: Non-research-paper materials (e.g., research notes or isolated preprints, scripts, or datasets) must reside on Figshare or other permissive repositories (e.g., the Open Science Framework). The author reserves standard-output sites (e.g., ResearchGate (RG), arXiv (AX), and OpenReview (OR)) for research-paper materials. A non-paper file may only qualify for standard-output sites if accompanied by a complete research paper draft.

5. If the file contains major changes and a completed research paper or its supporting materials but is not due for external review, place the file on F1, GH, and RG; otherwise, proceed to (6).

Reason: Setting ResearchGate as the single site for finished papers creates a filtered publication hub, which is useful for readers who seek only empirically supported claims. Concurrently, the RG upload process involves less friction than arXiv’s, where source files may be required and re-uploads/version adjustments are less straightforward.

6. If, in addition, the file is due for external review, place the file on the required review site (typically OR for major conferences) and on F1, GH, RG, and AX, subject to the venue’s preprint and anonymity policies.

Reason: A file must reside on the specified review site to qualify for peer review. arXiv serves as a public record of work the author considers ready for external review, and so needs updating. This is the final stage, and all repositories in the filter chain are updated simultaneously to reflect the current work. Authors and reviewers may then refer to the pre-publication OR/AX versions during the review process.

The current file, as a meta-policy document/research, utility operates outside of this policy. Research utilities belong on GitHub as a reference object for readers scrutinising the research process, but are fixed relative to the broader pipeline.

2 Research Categories

The author categorises files by their features. Features include quality, completeness, conciseness, and file type. One imagines the existence of a maximally comprehensive, orthogonal feature set $\{x_1, x_2, \dots, x_N\}$, yet distinguishing files according to these features may produce a categorisation that is hard to maintain and use. A categorisation policy C ’s utility, $U_{\text{researcher}}(C)$, thus increases with C ’s discriminative ability $u_d(C)$ and ease of maintenance/use $u_m(C)$.

$$U_{\text{researcher}}(C) = u_d(C) + u_m(C). \quad (1)$$

An optimal categorisation balances detail with naming/storage efficiency. The following scheme optimises towards C^* , but is only a heuristic predictor, $\hat{C} \neq C^*$. Figure 1 illustrates the categorisation decision tree; for efficiency, Table 1 defines only categories that currently contain outputs.

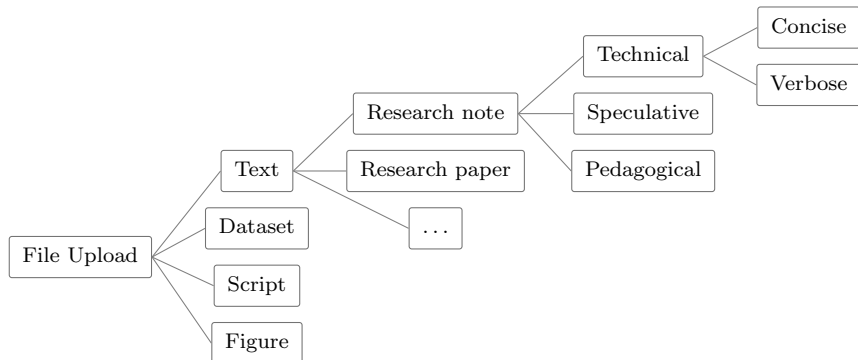


Figure 1: Research categorisation decision tree. Only currently used lower-level branches are expanded.

Table 1: Definitions of the currently non-empty research categories.

Level	Category	Description
File type	.pdf, .mp4, .py	The literal file type dictates how the author treats each output. For example, research notes/papers are typically LaTeX-written PDFs, and supplementary data or figures must exist concurrently with the .pdf for progression to ResearchGate, which the author reserves for papers only.
Advancement status	Research note	Research notes must contain an abstract, introduction/setup, main argument with a non-trivial result, and a conclusion. The format is otherwise unrestricted, and recent outputs (e.g., the h-bridge work) have distributed these core elements across a file series.
	Research paper	Research papers must contain the following sections: abstract, introduction/setup, literature review, data (if applied), methodology, results and discussion, conclusion, references, and appendices. A technical appendix must derive the main results or provide foundational constructions/assumptions for reproducibility.
Exposition standard	Technical	Technical outputs prioritise the formal statement, derivation, or empirical evaluation of a contribution. Outputs are assumed technical unless otherwise stated.
	Speculative	Speculative outputs develop conjectural or provisional ideas whose claims have not yet been fully derived or empirically validated. The label indicates their epistemic status rather than their file quality.
	Pedagogical	Pedagogical outputs prioritise accessible explanation and intuition while retaining the underlying technical content.
Conciseness/verbosity	Concise	Concise outputs present the central contribution and necessary support with minimal exposition.
	Verbose	Verbose outputs include extended explanations, derivations, examples, or contextual material beyond the concise form.

3 Programming and Code

Implementing experiments automatically via programming scripts is often the fastest and most precise approach to modern data analysis. Code is fast because computations can occur in parallel on optimised devices. Code is precise because syntactic or logical errors often force a program to halt; compilers *require* precision. This section articulates the author’s preferred approach to programming and code.

3.1 Modularity

Every experiment can be thought of as satisfying some requirements: data need fetching and cleaning, a model building, results plotting etc.. Typically, trying to satisfy an experiment’s entire requirement set with a single script is troublesome. Not only does this often effect runtime, but the mental cost of debugging and picturing the logic explodes with file length. Let the experiment E have script modules $\{m_1, m_2, \dots, m_{|M|}\}$ containing requirements $\{r_1, r_2, \dots, r_{|R|}\}$, let the placement policy be:

$$\pi_r : \{r_1, r_2, \dots, r_{|R|}\} \longrightarrow \mathcal{P}(\{m_1, m_2, \dots, m_{|M|}\}), \quad (2)$$

3.2 Version Types

Consider a bi-partite file-split. For each modularised code script, a concise and verbose version should exist. The concise version should house the minimal viable code to support the module’s requirements. The verbose version should include comments, deprecated snippets that remain relevant, and a version history log. So, the verbose version acts as a master file from which the concise version derives.

Table 2: General Feature Guide for Concise vs. Verbose Versions.

Feature	Concise	Verbose
Comments	Concise-version comments add necessary detail that the reader cannot glean from the code’s context or the cell headers. Omit comments that duplicate the code. Remove unnecessary punctuation from the end of each sentence. Allow sentence fragments and standalone words/phrases.	Verbose-version comments are pedagogical in nature; they direct the reader’s attention and explain the code. Sentences should be grammatically/syntactically correct. For navigability, each block should start with an orientation comment. Parenthetical remarks should appear in-line after the command ends.
Variable Names	Concise variable names should approach the minimal distinguishing limit. Single-character varnames for mathematical code are preferred unless the context dictates otherwise.	Verbose varnames may be longer than the minimal limit, but generally handling both the concise and verbose files with differing varnames is onerous; prioritise brevity.
In-file Logical Modularity	Place imports once at the script’s start. Restate definitions as little as possible. Do not duplicate definitions unless the call site requires self-sufficiency	-